

Final Evolution Lab — Testing Guide

Version: 1.0.0 | **Date:** April 2, 2026

Quick Start

```
# 1. Start all services
cd /home/ubuntu/rork-final-evolution-lab
./scripts/start_services.sh

# 2. Run automated tests
./scripts/test_deployment.sh

# 3. Open web player
open http://localhost:3000
```

1. Test Environment Setup

1.1 Prerequisites

Requirement	Minimum	Recommended
Node.js	18.x	20.x
GPU	Any NVIDIA	V100/A100
RAM	16 GB	64 GB
Storage	50 GB free	200 GB free
Browser	Chrome 100+	Chrome 120+
Network	10 Mbps	50 Mbps

1.2 Start Services

```
# Start signalling server + frontend
./scripts/start_services.sh

# Verify services
curl http://localhost:8888/healthz      # Should return {"status":"ok"}
curl http://localhost:3000             # Should return HTML
```

1.3 Start UE5 Server (if build complete)

```
source .ue5_env
cd Builds/LinuxServer/
./FinalEvolutionLabServer.sh \
  -PixelStreamingIP=127.0.0.1 \
  -PixelStreamingPort=8888 \
  -RenderOffscreen \
  -ForceRes -ResX=1920 -ResY=1080 \
  -FPS=60
```

2. Automated Test Suite

2.1 Full Deployment Test

```
./scripts/test_deployment.sh
```

Checks performed:

- ☒ Signalling server health (/healthz)
- ☒ Game modes API (/api/modes) — verifies 17 modes
- ☒ Web frontend accessibility
- ☒ WebSocket connectivity
- ☒ TURN/STUN server reachability
- ☒ Docker container status
- ☒ Build artifact existence

2.2 Game Modes Test

```
./scripts/test_game_modes.sh
```

Verifies all 17 modes are registered:

- basketball_h2h, basketball_dunk, basketball_3v3
 - karate_h2h, karate_endless
 - baseball, football, soccer
 - golf, tennis, volleyball
 - gymnastics, brain_brawl
 - surfing, skateboarding, snowboarding
 - market_browse
-

3. Manual Test Scenarios

3.1 Web Player Testing

#	Test Case	Steps	Expected Result
1	Load web player	Navigate to http://localhost:3000	Landing page loads with game mode selection
2	Mode selection	Click any game mode card	Mode details panel opens
3	Launch mode	Click “Play” on a mode	WebRTC connection initiates, stream starts
4	Game controls	Use keyboard/mouse in stream	Controls respond in UE5 game
5	Exit mode	Click exit/back button	Returns to mode selection
6	Responsive design	Resize browser to mobile width	UI adapts properly
7	Connection recovery	Disconnect WiFi, reconnect	Stream reconnects automatically

3.2 Game Mode Testing (Per Mode)

For each of the 17 game modes, test:

Basketball Modes

Test	basketball_h2h	basketball_dunk	basketball_3v3
Mode loads	✓ Venice Beach court	✓ Venice Beach court	✓ Venice Beach court
Player movement	WASD + mouse	WASD + mouse	WASD + mouse
Core mechanic	1v1 scoring	Dunk attempts	3v3 team play
Scoring works	Points counted	Style scoring	Team score
Animation plays	Running, jumping	Dunk animations	Pass, shoot

Combat Modes

Test	karate_h2h	karate_endless
Mode loads	✓ Dojo environment	✓ Dojo environment
Attack mechanics	Punch, kick, block	Continuous waves
AI opponent	Responds to attacks	Increasing difficulty

Field Sports

Test	baseball	football	soccer
Mode loads	✓ Ballpark	✓ Gridiron	✓ Stadium
Core mechanic	Pitch/bat	Kick return	Full match

Other Sports

Test	golf	tennis	volleyball	gymnastics
Mode loads	✓ Links	✓ Court	✓ Sand Court	✓ Floor
Core mechanic	Swing	Rally	Serve/spike	Routine

Board Sports & Special

Test	surfing	skateboard- ing	snowboard- ing	brain_brawl	mar- ket_browse
Mode loads	✓ Beach	✓ Park	✓ Mountain	✓ Arena	✓ Shop
Core mechanic	Wave ride	Trick chain	Downhill	Quiz battle	Browse items

3.3 Exercise System Testing

Test each exercise category:

Category: Warm-Up & Activation (4 exercises)

- ☐ Exercise loads with 3D animation
- ☐ Animation plays correctly
- ☐ Timer/rep counter works
- ☐ Can skip/advance

Category: Strength & Power (4 exercises)

- ☐ Exercise loads with 3D animation
- ☐ Proper form demonstrated
- ☐ Weight/resistance shown

Category: Mobility & Flexibility (4 exercises)

- ☐ Smooth animation transitions
- ☐ Hold timer counts down

Category: Sport-Specific Drills (8 exercises)

- ☐ Sport-appropriate environment loads
- ☐ Drill instructions clear

Category: Recovery & Cooldown (3 exercises)

- ☐ Calm animation style
- ☐ Breathing cues shown

3.4 Pixel Streaming Testing

#	Test	Steps	Expected
1	Stream quality	Launch any mode	1080p @ 60fps stream
2	Latency	Move mouse quickly	< 100ms input latency
3	Bitrate adaptation	Throttle network	Quality degrades gracefully
4	Multi-viewer	Open 2 browser tabs	Each gets unique session
5	Mobile stream	Open on phone browser	Touch controls work

3.5 iOS App Testing

Prerequisites:

- macOS with Xcode 15+
- iPhone with iOS 16+
- Apple Developer account

Test Checklist:

- ☐ App launches successfully
- ☐ Pixel Streaming connects
- ☐ Touch controls responsive
- ☐ Game modes accessible
- ☐ HealthKit permission prompt
- ☐ Camera permission prompt
- ☐ Motion permission prompt
- ☐ Background/foreground transitions
- ☐ Network disconnection handling
- ☐ Portrait **and** landscape modes

4. Performance Benchmarks

4.1 Target Metrics

Metric	Target	Acceptable
Server FPS	60 fps	30 fps
Stream latency	< 50ms	< 150ms
Page load time	< 2s	< 5s
WebSocket connect	< 500ms	< 2s
Mode switch time	< 3s	< 8s
Memory usage	< 8 GB	< 16 GB
GPU utilization	40-80%	< 95%

4.2 Monitoring Commands

```
# GPU monitoring
nvidia-smi -l 5

# Server FPS
grep "FPS" logs/ue5_server.log | tail -10

# Network stats
ss -s

# Memory usage
free -h

# Process monitoring
htop -p $(pgrep -f FinalEvolution)
```

5. API Testing

5.1 Signalling Server API

```
# Health check
curl http://localhost:8888/healthz
# Expected: {"status":"ok"}

# Get game modes
curl http://localhost:8888/api/modes
# Expected: JSON with 17 modes

# WebSocket test (Python)
python3 -c "
import asyncio, websockets, json
async def test():
    async with websockets.connect('ws://localhost:8889') as ws:
        msg = json.loads(await ws.recv())
        print(f'Connected: {msg}')
        assert msg['type'] == 'playerConnected'
        print('✅ WebSocket test passed')
asyncio.run(test())
"
```

5.2 AI Asset Pipeline API

```
# Verify asset manifest
python3 -c "
import json
with open('GeneratedAssets/Environments/openart_asset_manifest.json') as f:
    d = json.load(f)
    envs = d.get('environments', {})
    print(f'Environments: {len(envs)}')
    for name in envs:
        print(f'  ✓ {name}')
"

# Verify animation manifest
python3 -c "
import json
with open('GeneratedAssets/Animations/animation_manifest.json') as f:
    d = json.load(f)
    print(f'Custom animations: {d["statistics"]["total_custom_animations"]}')
"
```

6. Issue Reporting

6.1 How to Report

When reporting issues, include:

1. **Environment:** OS, browser, device
2. **Steps to reproduce:** Exact sequence
3. **Expected behavior:** What should happen
4. **Actual behavior:** What happened
5. **Screenshots/video:** If visual
6. **Console logs:** Browser dev tools (F12)
7. **Server logs:** `tail -100 logs/build_output.log`

6.2 Log Locations

Log	Path
Build output	<code>logs/build_output.log</code>
Signalling server	<code>logs/signalling.log</code>
Frontend	<code>logs/frontend.log</code>
UE5 Server	<code>Saved/Logs/FinalEvolutionLab.log</code>
Asset pipeline	<code>GeneratedAssets/pipeline_report.json</code>

6.3 Common Test Failures

Failure	Cause	Fix
"Connection refused" on : 8888	Signalling not running	<code>./scripts/start_services.sh</code>
"No streamer connected"	UE5 server not running	Start UE5 server first
Black stream screen	GPU not rendering	Check <code>nvidia-smi</code> , verify <code>-RenderOffscreen</code>
Mode not found	Server not updated	Restart signalling server
WebSocket timeout	Firewall blocking	Open ports 8888, 8889, 3478
iOS app crash	Signing invalid	Re-sign with valid certificate

7. Test Environment Teardown

```
# Stop all services
./scripts/stop_services.sh

# Stop Docker containers
docker-compose -f streaming/docker-compose.yml down

# Kill any remaining processes
pkill -f "FinalEvolution"
pkill -f "signalling"
```